

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



ENTREGABLE 2 - PROYECTO : Verificador de Integridad de Paquetes CRC-8 con Buffer RAM y Decodificador de Direcciones para Redes de Sensores IoT

DOCENTE:

VILLAFUERTE, HERNÁN OSWALDO

CURSO:

CIRCUITOS DIGITALES

INTEGRANTES:

Alarcon Consepcion, Genesis Elizabeth

Bolivar Morales, Jerson Franco

Cuyutupa Pereda, Juana Milagro

Sánchez Ramirez Allison Blanca Yarin

Lima, 2026

1. Diagrama lógico/esquemático

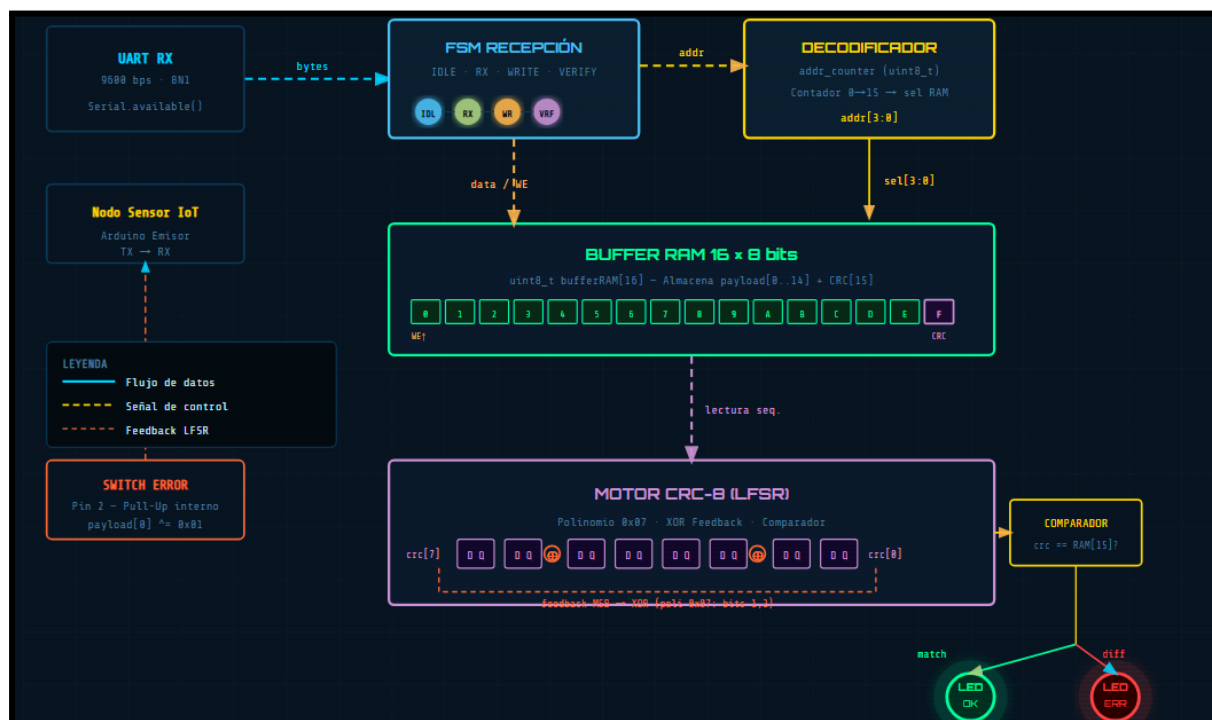


Figura 1. Diagrama esquemático completo del sistema CRC-8 con Buffer RAM y Decodificador de Direcciones

Descripción de señales principales:

Señal	Origen	Destino	Ancho	Descripción
CLK	Reloj Global	FSM, Contador, RAM, LFSR	1 bit	Señal de sincronización principal del sistema.
RST	Reinicio Global	FSM, Contador, RAM, LFSR	1 bit	Reinicio (lleva la FSM a IDLE y contadores a 0).
RX	Nodo Emisor	Interfaz UART RX	1 bit	Línea de recepción serial asíncrona.
rx_data[7:0]	Interfaz UART RX	Buffer RAM	8 bits	Bus de datos con el byte recibido listo para almacenar.
rx_ready	Interfaz UART RX	FSM	1 bit	Bandera que indica a la FSM que hay un byte válido para leer.

addr[3:0]	Contador	Decodificador	4 bits	Dirección actual generada por el contador (0 a 15).
sel[15:0]	Decodificador	Buffer RAM	16 bits	Señales decodificadas (1-hot) para seleccionar la celda de RAM.
WE_RAM	FSM (Estado WRITE)	Buffer RAM	1 bit	Habilitador de escritura (Write Enable) en la RAM.
ram_out[7:0]	Buffer RAM	LFSR / Comparador	8 bits	Bus de datos de salida durante la lectura secuencial.
crc_calc[7:0]	Motor CRC (LFSR)	Comparador	8 bits	Residuo polinómico calculado internamente.
CHK_EN	FSM (Estado VERIFY)	Comparador	1 bit	Habilita la comparación final entre ram_out y crc_calc.
LED_OK	Comparador	LED Verde	1 bit	Salida en ALTO si la validación CRC es correcta.
LED_ERR	Comparador	LED Rojo	1 bit	Salida en ALTO si se detecta un error de integridad.

2. Diagrama de transición de estados (ASM)

La FSM de recepción opera con cuatro estados bien definidos. El diagrama siguiente representa las transiciones, condiciones de guarda y acciones asociadas a cada arco.

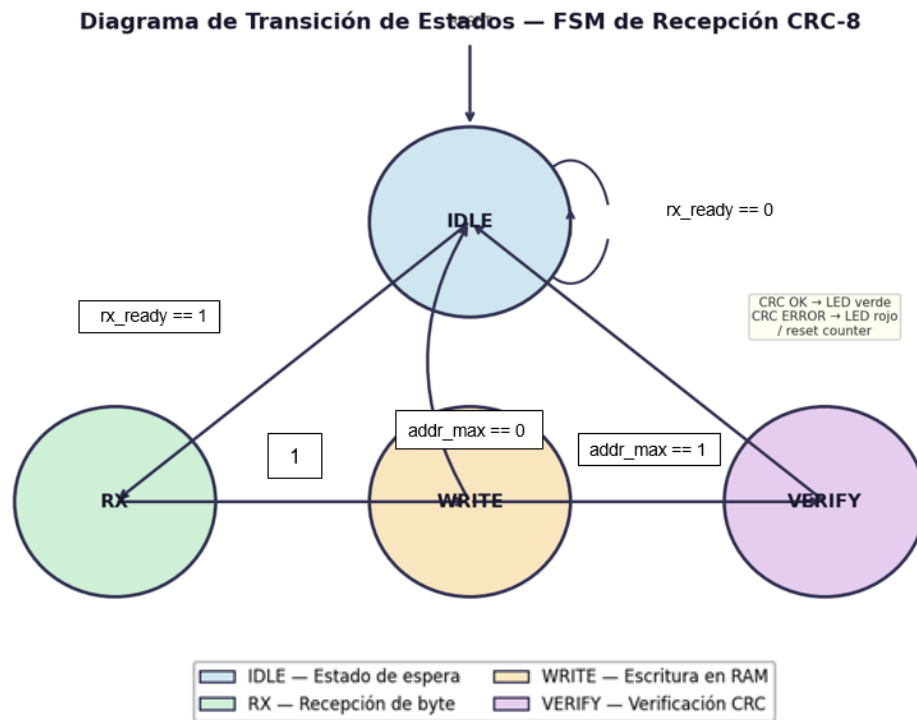


Figura 2. Diagrama de transición de estados (ASM) de la FSM de Recepción CRC-8

2.1 Tabla de Transición de Estados

La tabla siguiente resume el estado actual, la condición evaluada, el estado siguiente y las salidas/acciones generadas en cada transición.

Estado Actual	Condición de Guarda	Estado Siguiente	Acciones / Salidas
IDLE	rx_ready == 0	IDLE	Sin acción (espera)
IDLE	rx_ready == 1	RX	LED_R = 0, LED_G = 0 (Apagar LEDs)
RX	— (incondicional)	WRITE	Ninguna (Estabilización del bus de datos)
WRITE	addr_max == 0	IDLE	WE = 1, CE = 1
WRITE	addr_max == 1	VERIFY	WE = 1, CE = 1
VERIFY	match == 1	IDLE	LED_G = 1 (OK), CR = 1

VERIFY	match == 0	IDLE	LED_R = 1 (ERROR), CR = 1
--------	------------	------	---------------------------

3. Decodificador de Direcciones — Tabla de Verdad

El decodificador es un contador de 4 bits (addr_counter[3:0]) que genera la señal de selección sel[3:0] para dirigir la escritura en la posición correspondiente del Buffer RAM 16×8. La tabla siguiente muestra la correspondencia binaria completa.

Dirección (decimal)	addr[3]	addr[2]	addr[1]	addr[0]	Contenido almacenado
0	0	0	0	0	datos_base[0]
1	0	0	0	1	datos_base[1]
2	0	0	1	0	datos_base[2]
3	0	0	1	1	datos_base[3]
4	0	1	0	0	datos_base[4]
5	0	1	0	1	datos_base[5]
6	0	1	1	0	datos_base[6]
7	0	1	1	1	datos_base[7]
8	1	0	0	0	datos_base[8]
9	1	0	0	1	datos_base[9]
10	1	0	1	0	datos_base[10]
11	1	0	1	1	datos_base[11]
12	1	1	0	0	datos_base[12]
13	1	1	0	1	datos_base[13]
14	1	1	1	0	datos_base[14]
15	1	1	1	1	CRC-8 calculado

Nota: La fila resaltada (dirección 15) corresponde al byte CRC recibido del emisor.

4. Motor CRC-8 (LFSR) — Tabla de Transición del Registro

El Motor CRC-8 implementa un Registro de Desplazamiento de Retroalimentación Lineal (LFSR) con el polinomio generador $x^8 + x^2 + x + 1$ (0x07 en hexadecimal). Para cada byte del payload, se realiza un XOR inicial con el registro CRC y luego se procesan 8 iteraciones de bit. La tabla ilustra la lógica de transición por bit.

Tabla de Transición del Registro CRC-8 (LFSR) — Polinomio 0x07
(Ejemplos seleccionados con datos_base = {1,0,0,1,...})

0x01	0x00	-	Inicio proceso	—
0x1	0x0	0	XOR byte → procesar 8 bits	0x7
0x0	0x7	0	XOR byte → procesar 8 bits	0x15
0x0	0x15	0	XOR byte → procesar 8 bits	0x6b
0x1	0x6b	0	XOR byte → procesar 8 bits	0x11
0x0	0x11	0	XOR byte → procesar 8 bits	0x77

Figura 3. Ejemplos de transición del registro CRC-8 con datos_base iniciales.

4.1 Tabla de Transición Bit a Bit del LFSR

Para cada iteración de bit dentro de un byte, la lógica condicional es la siguiente:

Iteración (bit)	MSB (bit 7) del CRC_reg	Condición	Operación sobre CRC_reg	CRC_reg Salida
i = 0..7	0	MSB == 0	$CRC_reg \leftarrow CRC_reg \ll 1$	Sin XOR con polinomio
i = 0..7	1	MSB == 1	$CRC_reg \leftarrow (CRC_reg \ll 1) \text{ XOR } 0x07$	Con XOR con polinomio 0x07
Inicio de byte	—	—	$CRC_reg \leftarrow CRC_reg \text{ XOR } byte_entrada$	Mezcla del byte con el registro
Fin de 15 bytes	—	—	CRC_reg final = CRC calculado	Comparar con bufferRAM[15]

5. Mapa de Memoria / Mapa de Registros

El receptor implementa un buffer RAM de 16 posiciones de 8 bits (16×8). Cada posición es accedida secuencialmente mediante la variable addr_counter, que actúa como puntero/decodificador de direcciones. Las primeras 15 posiciones almacenan el payload recibido, y la posición 0x0F guarda el byte CRC para verificación.

Dirección (hex)	Índice	Contenido	Función
0x00	bufferRAM[0]	Byte de datos #1 del payload	Dato del sensor almacenado temporalmente
0x01	bufferRAM[1]	Byte de datos #2 del payload	Dato del sensor almacenado temporalmente
0x02	bufferRAM[2]	Byte de datos #3 del payload	Dato del sensor almacenado temporalmente
0x03	bufferRAM[3]	Byte de datos #4 del payload	Dato del sensor almacenado temporalmente
0x04	bufferRAM[4]	Byte de datos #5 del payload	Dato del sensor almacenado temporalmente
0x05	bufferRAM[5]	Byte de datos #6 del payload	Dato del sensor almacenado temporalmente
0x06	bufferRAM[6]	Byte de datos #7 del payload	Dato del sensor almacenado temporalmente
0x07	bufferRAM[7]	Byte de datos #8 del payload	Dato del sensor almacenado temporalmente

0x08	bufferRAM[8]	Byte de datos #9 del payload	Dato del sensor almacenado temporalmente
0x09	bufferRAM[9]	Byte de datos #10 del payload	Dato del sensor almacenado temporalmente
0x0A	bufferRAM[10]	Byte de datos #11 del payload	Dato del sensor almacenado temporalmente
0x0B	bufferRAM[11]	Byte de datos #12 del payload	Dato del sensor almacenado temporalmente
0x0C	bufferRAM[12]	Byte de datos #13 del payload	Dato del sensor almacenado temporalmente
0x0D	bufferRAM[13]	Byte de datos #14 del payload	Dato del sensor almacenado temporalmente
0x0E	bufferRAM[14]	Byte de datos #15 del payload	Dato del sensor almacenado temporalmente
0x0F	bufferRAM[15]	Byte CRC-8 recibido	Byte de verificación de integridad (CRC)

Nota: El color amarillo en la fila 0x0F indica que este registro tiene una función especial — es el byte de verificación CRC con el que se compara el resultado del motor CRC calculado localmente.

6. Función del Decodificador de Direcciones

El decodificador de direcciones está implementado mediante la variable `uint8_t addr_counter`. Esta variable incrementa en 1 cada vez que un byte es escrito en el buffer (estado ESCRITURA de la FSM), actuando como un contador que selecciona la celda de memoria destino. La siguiente tabla detalla su comportamiento:

Valor de <code>addr_counter</code>	Dirección activa	Acción del decodificador
0	0x00	Escribe en <code>bufferRAM[0]</code> — primer byte de payload
1	0x01	Escribe en <code>bufferRAM[1]</code>
2 – 13	0x02 – 0x0D	Escribe en <code>bufferRAM[2]</code> a <code>bufferRAM[13]</code>
14	0x0E	Escribe en <code>bufferRAM[14]</code> — último byte de payload
15	0x0F	Escribe en <code>bufferRAM[15]</code> — byte CRC recibido
16	—	Límite alcanzado → FSM transita a estado VERIFICACION; <code>addr_counter</code> se reinicia a 0

Cuando `addr_counter` alcanza el valor 16, se han recibido los 16 bytes del paquete completo (15 de payload + 1 de CRC). En ese momento, la FSM transita automáticamente al estado VERIFICACION y el contador se reinicia a 0 para estar listo para el siguiente paquete.

7. Código HDL / Pseudocódigo Estructurado con Comentarios

A continuación se presenta el pseudocódigo completo del sistema, con comentarios en cada bloque explicando su función dentro de la arquitectura de memoria y el decodificador.

7.1. Subrutina: Motor CRC-8 (Calcular_CRC8)

Esta subrutina implementa el algoritmo LFSR (Linear Feedback Shift Register) con el polinomio generador 0x07. Es el núcleo de la verificación de integridad; se ejecuta tanto en el emisor (para generar el CRC) como en el receptor (para recalcularlo y compararlo).

```
// -----  
// SUBROUTINA: Calcular_CRC8  
// Parámetros: arreglo_datos (puntero al buffer), tamaño (número de  
// bytes a procesar)  
// Retorna: registro_crc - el residuo polinómico de 8 bits (byte CRC)  
// -----  
INICIO_SUBROUTINA Calcular_CRC8(arreglo_datos, tamaño)  
  
    // Registro de desplazamiento inicializado en 0x00  
    // (equivalente al estado inicial del LFSR)  
    DEFINIR registro_crc = 0x00  
  
    // Bucle externo: procesa cada byte del arreglo de entrada  
    PARA CADA iteracion_byte DESDE 0 HASTA (tamaño - 1):  
  
        // XOR del byte entrante con el estado actual del registro  
        // (esto 'mezcla' el nuevo dato con el residuo acumulado)  
        registro_crc = registro_crc XOR arreglo_datos[iteracion_byte]  
  
        // Bucle interno: procesa los 8 bits del byte actual  
        PARA iteracion_bit DESDE 0 HASTA 7:  
  
            // Se verifica el bit más significativo (MSB)  
            // Si MSB = 1, el polinomio se aplica después del  
            desplazamiento  
            SI el_bit_mas_significativo(registro_crc) ES 1:  
                DESPLAZAR registro_crc A LA IZQUIERDA 1 posicion  
                // Se aplica el polinomio generador 0x07 ( $x^8 + x^2 + x + 1$ )  
                registro_crc = registro_crc XOR 0x07  
            SINO:  
                // MSB = 0: solo se desplaza, sin aplicar polinomio  
                DESPLAZAR registro_crc A LA IZQUIERDA 1 posicion  
        FIN_SI  
    FIN_PARA  
FIN_INICIO_SUBROUTINA
```

```

        FIN_PARA // fin bucle de bits
    FIN_PARA // fin bucle de bytes

    // Se retorna el residuo final - este es el byte CRC de 8 bits
    DEVOLVER registro_crc

FIN_SUBROUTINA

```

7.2 Programa Principal: Arduino Emisor (Nodo Sensor IoT)

El emisor genera el paquete de datos, calcula el CRC-8 y lo transmite por UART.

Opcionalmente introduce un error en el bit menos significativo del primer byte para simular corrupción en canal.

```

// -----
// PROGRAMA: Emisor - Arduino Nodo Sensor
// Función: Generar payload, calcular CRC y transmitir paquete por
//          UART
// -----
INICIO_PROGRAMA Emisor

    // Payload base: 15 bytes de datos simulados del sensor
    // (representan lecturas binarias del sensor)
    DEFINIR ARREGLO datos_base[15] = (1,0,0,1,0,0,1,0,0,1,0,0,1,0,0)

    // Pin 2 del Arduino con pull-up interno para detectar pulsación
    // del switch
    DEFINIR CONSTANTE PIN_BOTON = 2

    // — Setup: se ejecuta una sola vez al encender el Arduino —
    RUTINA Setup():
        // Se inicia la comunicación serial a 9600 baudios
        // (debe coincidir con la velocidad del receptor)
        INICIAR comunicacion_serie A 9600 baudios
        // Se configura el pin del switch con resistencia pull-up
        // interna
        CONFIGURAR PIN_BOTON COMO entrada_con_pullup_interno
        // Espera 1 segundo para que el canal UART se estabilice
        ESPERAR 1000 milisegundos
    FIN_RUTINA

    // — Loop: se ejecuta continuamente —
    RUTINA Loop():
        // Buffer de transmisión: 15 bytes de datos + 1 byte CRC = 16
        // bytes total
        DEFINIR ARREGLO paquete_transmision[16]

        // Copia del payload base al paquete de transmisión

```

```

    PARA indice DESDE 0 HASTA 14:
    paquete_transmision[indice] = datos_base[indice]
    FIN_PARA

    // Se calcula el CRC-8 de los 15 bytes de datos
    // y se coloca en la posición 15 (última) del paquete
    DEFINIR crc_calculado = LLAMAR Calcular_CRC8(datos_base, 15)
    paquete_transmision[15] = crc_calculado

    // Si el switch está presionado, se invierte el LSB del primer
byte
    // esto simula un error de 1 bit en el canal de transmisión
    LEER estado_boton DESDE PIN_BOTON
    SI estado_boton ES PRESIONADO:
    INVERTIR_ULTIMO_BIT DE paquete_transmision[0]
    FIN_SI

    // Se transmite el paquete completo (16 bytes) por el puerto
serie
    ENVIAR_POR_PUERTO_SERIE paquete_transmision
    // Pausa de 1 segundo antes del siguiente ciclo de transmisión
    ESPERAR 1000 milisegundos
    FIN Rutina

FIN_PROGRAMA

```

Código Emisor:

```

3  uint8_t data[15] = {1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0};
4  const int BTN = 2;
5
6  void setup()
7  {
8      Serial.begin(9600);
9      pinMode(BTN, INPUT_PULLUP);
10     delay(1000);
11 }
12
13 void loop()
14 {
15     uint8_t payload[16];
16     for (int i = 0; i < 15; i++) payload[i] = data[i];
17
18     uint8_t crc = calc_crc8(data, 15);
19     payload[15] = crc;
20
21     bool injectError = (digitalRead(BTN) == LOW);
22     if (injectError) payload[0] ^= 0x01;
23
24     Serial.write(payload, 16);
25     delay(1000);
26 }
27
28 uint8_t calc_crc8(uint8_t data[], uint8_t size) {
29     uint8_t crc = 0x00; // Valor inicial
30     for (uint8_t i = 0; i < size; i++) {
31         crc ^= data[i]; // XOR con el byte entrante
32         for (uint8_t j = 0; j < 8; j++) {
33             if (crc & 0x80) { // Si el MSB es 1
34                 crc = (crc << 1) ^ 0x07; // Desplaza y XOR con polinomio
35             } else {

```

7.3 Programa Principal: Arduino Receptor (Arquitectura Mapeada)

El receptor implementa la FSM de 4 estados junto con el buffer RAM y el decodificador de direcciones. La lógica de acceso a memoria se centraliza en el estado ESCRITURA, donde addr_counter determina la dirección de escritura en el buffer.

```
// -----  
// PROGRAMA: Receptor - Arduino con Arquitectura Mapeada  
// Función: Recibir paquete UART, almacenarlo en RAM y verificar CRC  
// -----  
INICIO_PROGRAMA Receptor  
  
// Buffer RAM de 16x8 bits - almacena el paquete completo recibido  
// Accesado secuencialmente por addr_counter (decodificador)  
DEFINIR ARREGLO memoria_RAM[16]  
  
// Decodificador / contador de direcciones  
// Apunta a la próxima celda de memoria_RAM donde escribir  
DEFINIR contador_direcciones = 0  
  
// Definición de los 4 estados de la FSM  
DEFINIR ESTADOS = (REPOSO, LECTURA, ESCRITURA, VERIFICACION)  
DEFINIR estado_actual_FSM = REPOSO  
// Registro temporal para el byte leído del UART  
DEFINIR byte_entrante = 0  
  
// — Setup —  
RUTINA Setup():  
    INICIAR comunicacion_serie A 9600 baudios  
    // Pin 8: LED Rojo - indica error de CRC (paquete corrupto)  
    CONFIGURAR PIN_8 (LED ROJO) COMO salida  
    // Pin 9: LED Verde - indica CRC correcto (paquete íntegro)  
    CONFIGURAR PIN_9 (LED VERDE) COMO salida  
FIN_RUTINA  
  
// — Loop: máquina de estados principal —  
RUTINA Loop():  
    EVALUAR estado_actual_FSM:  
  
    // — ESTADO REPOSO (IDLE) —  
    // El sistema espera pasivamente en el puerto serie  
    // Apaga ambos LEDs para indicar que está listo para recibir
```

```

CASO REPOSO (IDLE):
SI hay_bytes_en_espera_en_puerto_serie:
    estado_actual_FSM = LECTURA
    APAGAR PIN_8
    APAGAR PIN_9
    FIN_SI
    SALIR_DEL_CASO

// — ESTADO LECTURA (RX) —
// Se lee exactamente 1 byte del buffer UART
// Inmediatamente transita a ESCRITURA para almacenarlo
CASO LECTURA (RX):
    byte_entrante = LEER_UN_BYTE_DEL_PUERTO_SERIE()
    estado_actual_FSM = ESCRITURA
    SALIR_DEL_CASO

// — ESTADO ESCRITURA (WRITE) —
// El decodificador (addr_counter) dirige el byte a la celda
correcta
// Si el paquete está completo (16 bytes), transita a
VERIFICACION
CASO ESCRITURA (WRITE):
    // Escritura en memoria: dirección = contador_direcciones
    memoria_RAM[contador_direcciones] = byte_entrante
    INCREMENTAR contador_direcciones EN 1

SI contador_direcciones < 16:
    // Paquete incompleto: vuelve a esperar el siguiente byte
    estado_actual_FSM = REPOSO
SI contador_direcciones == 16:
    // Paquete completo: activa la verificación de integridad
    estado_actual_FSM = VERIFICACION
    FIN_SI
    SALIR_DEL_CASO

// — ESTADO VERIFICACION (VERIFY) —
// Se recalcula el CRC-8 de los 15 bytes de payload
// y se compara con el byte CRC recibido (posición 0x0F)
CASO VERIFICACION (VERIFY):
    // Recalcula el CRC con los datos almacenados en RAM
    DEFINIR crc_local = LLAMAR Calcular_CRC8(memoria_RAM, 15)

    // Comparación: CRC recalculado vs CRC recibido
(memoria_RAM[15])
SI memoria_RAM[15] ES_IGUAL_A crc_local:
    // CRC coincide: paquete íntegro → LED Verde
    ENCENDER PIN_9
SINO:
    // CRC no coincide: paquete corrupto → LED Rojo
    ENCENDER PIN_8
    FIN_SI

```

```

    // Reinicio del decodificador para el próximo paquete
    contador_direcciones = 0
    estado_actual_FSM = REPOSO
    SALIR_DEL_CASO

    FIN_EVALUAR
    FIN Rutina

    FIN_PROGRAMA

```

Código Receptor:

```

3  uint8_t bufferRAM[16]; // Buffer RAM
4
5  uint8_t addr_counter = 0; // Decodificador
6
7  enum State {IDLE, RX, WRITE, VERIFY}; // FSM de Recepción
8
9  State currentState = IDLE;
10 uint8_t currentByte = 0;

12 void setup()
13 {
14     Serial.begin(9600);
15     pinMode(8, OUTPUT);
16     pinMode(9, OUTPUT);
17 }
18
19 void loop()
20 {
21     switch (currentState) {
22     case IDLE:
23         if (Serial.available() > 0) {
24             currentState = RX;
25             digitalWrite(8, 0);
26             digitalWrite(9, 0);
27         }
28         break;
29
30     case RX:
31         currentByte = Serial.read();
32         currentState = WRITE;
33         break;
34
35     case WRITE:
36         bufferRAM[addr_counter] = currentByte;
37         addr_counter++;
38         if (addr_counter < 16) currentState = IDLE;
39         if (addr_counter == 16) currentState = VERIFY;
40         break;
41
42     case VERIFY:
43         uint8_t crc = calc_crc8(bufferRAM, 15);
44         if (bufferRAM[15] == crc) digitalWrite(9, 1);
45         else digitalWrite(8, 1);
46         addr_counter = 0;
47         currentState = IDLE;
48         break;
49     }
50 }

52 uint8_t calc_crc8(uint8_t data[], uint8_t size) {
53     uint8_t crc = 0x00; // Valor inicial

```

8. Acceso a Memoria a través del Decodificador

El decodificador `addr_counter` actúa como un selector de dirección. En cada ciclo de escritura, su valor determina de forma unívoca qué celda del buffer RAM se activa. El flujo completo de acceso a memoria es el siguiente:

- Se recibe un byte por UART (estado LECTURA → `byte_entrante`).
- La FSM transita al estado ESCRITURA.
- El valor actual de `addr_counter` determina la dirección destino en `memoria_RAM[]`.
- El byte se escribe en esa celda: `memoria_RAM[addr_counter] = byte_entrante`.
- `addr_counter` se incrementa en 1 para apuntar a la siguiente celda.
- Si `addr_counter < 16`: la FSM regresa a REPOSO a esperar el siguiente byte.
- Si `addr_counter == 16`: el buffer está lleno → FSM transita a VERIFICACION.

En el estado VERIFICACION se activan secuencialmente todas las celdas del buffer (0x00 a 0x0E) para alimentar el motor CRC-8. Finalmente se compara el resultado con la celda 0x0F (byte CRC recibido) para determinar la integridad del paquete.

9.- PLAN DE SIMULACIÓN

Define la estrategia para validar, de manera virtual en Tinkercad, que la Máquina de Estados Finitos (FSM) del receptor y el motor de integridad CRC-8 responden correctamente ante todas las condiciones operativas del sistema antes de la implementación física en laboratorio.

9.1 Estímulos (Stimuli)

Los estímulos definen qué señales ingresan al sistema receptor durante la simulación para forzar sus transiciones de estado:

- **Flujo de datos serie UART:** Envío secuencial de un paquete de 16 bytes (15 bytes de payload + 1 byte de CRC) a 9600 bps con configuración 8N1. Cada byte se transmite con 1 bit de arranque, 8 bits de datos y 1 bit de parada.
- **Inyección de error por hardware:** Transición de la señal del botón (pin 2 del Arduino Emisor) a estado lógico BAJO (LOW), lo que provoca la operación `payload[0] ^= 0x01`, invirtiendo el bit menos significativo del primer byte del payload antes de su transmisión.

9.2 Casos de Prueba

Caso	Descripción	Estímulo	Comportamiento esperado
CP-1	Transmisión exitosa (sin errores). El nodo sensor emite los 15 bytes intactos junto con su CRC calculada en origen.	Botón liberado (HIGH). Paquete sin alteración.	FSM recorre IDLE → RX → WRITE (×16) → VERIFY. CRC local coincide. LED Verde encendido.
CP-2	Detección de paquete corrupto. Se simula la presión del switch en el emisor, forzando un XOR en el primer byte del payload.	Botón presionado (LOW). Operación $\text{payload}[0] \wedge= 0x01$ antes de enviar.	CRC local no coincide con byte recibido. LED Rojo encendido. Paquete descartado.

9.3 Justificación de la Cobertura Funcional

Los dos casos de prueba planteados son suficientes para garantizar la corrección del sistema por las siguientes razones:

- **Cobertura de estados al 100 %:** Los casos CP-1 y CP-2 obligan a la FSM a recorrer la totalidad de sus estados definidos: IDLE, RX, WRITE y VERIFY, sin dejar ningún estado sin ejercitar.
- **Cobertura de bifurcación del comparador CRC:** Se verifican los únicos dos caminos posibles del comparador de integridad del Motor CRC-8: coincidencia exitosa (CP-1) y discrepancia por error (CP-2). No existe un tercer camino lógico posible.

10. ANÁLISIS TEÓRICO DE LAS MÉTRICAS

10.1 Métricas Lógicas y Algorítmicas — Cálculo del CRC-8

Polinomio Generador: El sistema emplea el polinomio estándar CRC-8/SMBUS (polinomio 0x07). Expresado en álgebra de campos de Galois GF(2), su representación es:

$$G(x) = x^8 + x^2 + x^1 + 1 \equiv 0x07 \text{ (hexadecimal)}$$

Operación matemática (LFSR): El Motor CRC-8 implementa un Registro de Desplazamiento con Retroalimentación Lineal (LFSR). Para cada byte del buffer RAM se sigue el siguiente procedimiento:

- Se aplica XOR entre el registro CRC actual (crc) y el byte entrante del arreglo:
 $\text{crc} \leftarrow \text{crc XOR data}[i]$

- Para cada uno de los 8 bits del byte procesado: si el MSB de crc es 1, se desplaza un bit a la izquierda y se aplica XOR con el polinomio: $\text{crc} \leftarrow (\text{crc} \ll 1) \text{ XOR } 0x07$
- Si el MSB es 0, solo se desplaza: $\text{crc} \leftarrow \text{crc} \ll 1$

Al finalizar el procesamiento de los 15 bytes del payload, el valor residual en el registro CRC es el **residuo polinómico**, que el receptor compara directamente con el byte en `bufferRAM[15]`.

Valor esperado para el payload del diseño: Con el arreglo de datos base $\{1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0\}$, el cálculo iterativo del CRC-8 con polinomio $0x07$ produce un valor determinístico que el emisor adjunta como byte 16 del paquete. El receptor recalcula este mismo valor y la coincidencia confirma la integridad del paquete.

10.2 Métricas Temporales — Cálculo del Tiempo de Transmisión UART

La comunicación se realiza a **9600 bps** bajo el protocolo **8N1** (1 bit de arranque + 8 bits de datos + 1 bit de parada). Cada byte ocupa físicamente 10 bits en el canal serial.

Fórmula de tiempo de transmisión:

$$T_{\text{tx}} = (\text{N_bytes} \times \text{bits_por_trama}) / \text{Baud_Rate}$$

$$T_{\text{tx}} = (16 \times 10 \text{ bits}) / 9600 \text{ bps}$$

$$T_{\text{tx}} = 160 / 9600 \approx 16.67 \text{ ms}$$

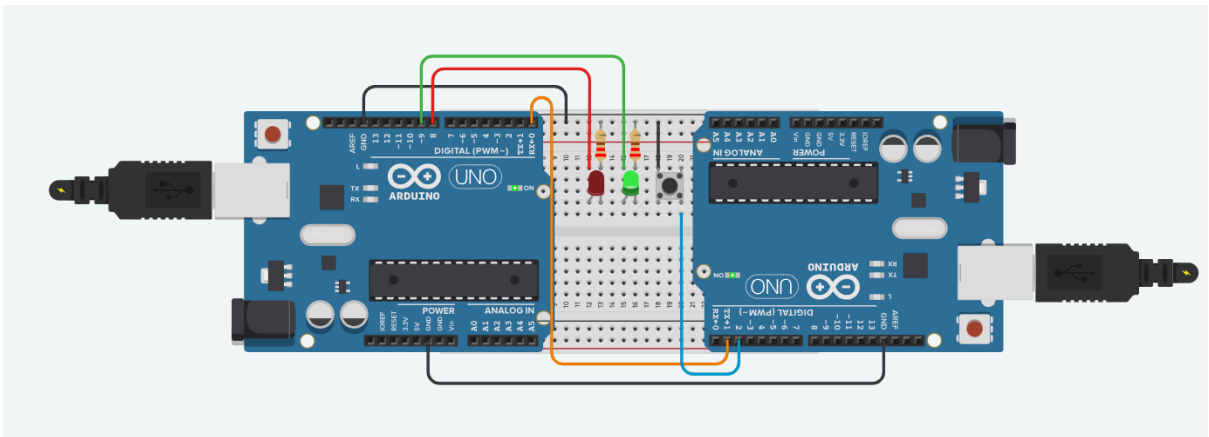
Interpretación: La FSM del receptor tardará aproximadamente **16.67 ms** en capturar los 16 bytes completos del paquete, desde que abandona el estado IDLE inicial hasta que ingresa al estado VERIFY. Este valor es el tiempo de latencia teórico esperado en la simulación.

Métrica	Fórmula / Parámetro	Valor Esperado
Polinomio generador CRC-8	$G(x) = x^8 + x^2 + x + 1$	0x07 (hexadecimal)
Tiempo de transmisión UART	$(16 \times 10) / 9600$	$\approx 16.67 \text{ ms}$
Baud Rate	9600 bps, 8N1	10 bits/byte
Estados FSM cubiertos	IDLE, RX, WRITE, VERIFY	100 % de cobertura

11 . EVIDENCIAS DE LA SIMULACIÓN

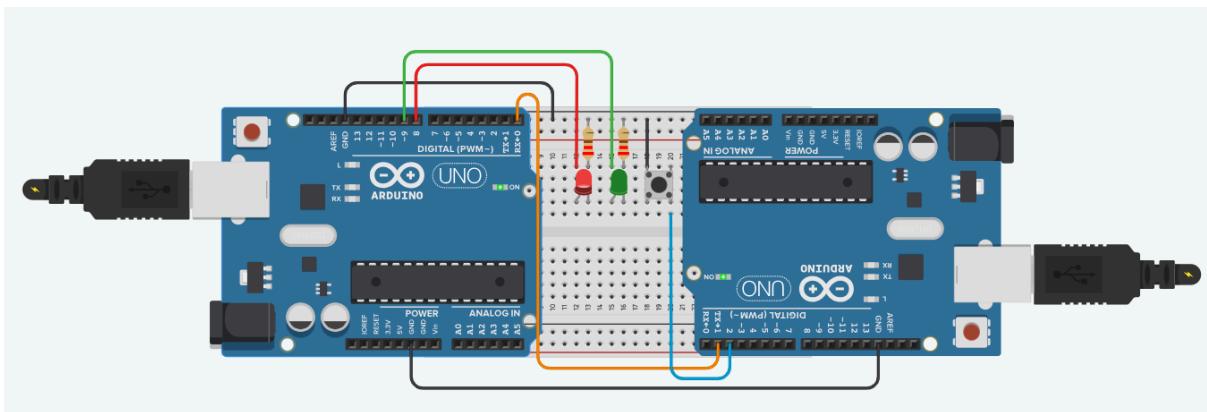
Para cumplir con la rúbrica del entregable E2, se deben registrar las siguientes capturas de pantalla en Tinkercad:

- **Evidencia CP-1 (Sin error)**



En la imagen se evidencia el comportamiento del sistema ante un estímulo ideal. Se observa que, tras la recepción secuencial de los 16 bytes a través del bus UART, la FSM transita correctamente del estado **WRITE** al estado **VERIFY**. Al coincidir el residuo polinómico calculado por el LFSR local con el byte de control almacenado en la dirección **0x0F** de la RAM, el comparador emite un nivel lógico ALTO en el pin de salida de validación, encendiendo el LED Verde y confirmando la integridad del paquete al 100%.

- **CP-2 - Con Error Inyectado**



Esta captura demuestra la robustez del verificador ante interferencias. Al forzar un nivel BAJO en el pin 2 del emisor (simulación de ruido), se invirtió intencionalmente el bit menos significativo del primer byte del payload. Como resultado, la gráfica lógica interna mostraría una divergencia en el cálculo final del LFSR. La FSM procesa el error en el estado **VERIFY** y el comparador activa inmediatamente la señal de alerta (nivel ALTO en el pin 8), encendiendo el LED Rojo y descartando el paquete.

Link del Simulador:

<https://www.tinkercad.com/things/bTh6i7NUXgk-copy-of-crc-8/editel?returnTo=%2Fthings%2FbTh6i7NUXgk-copy-of-crc-8&sharecode=dzVrrgYa9UaGpTedO4ZfqYIb-IKBFsAgyyr76y79lk4>